TECHNICAL ASSESSMENT

Resume Screening System: Design Brief

For the role of NLP Engineer

ISSUED BY	ProITbridge AI, Language Systems Group
ROLE	NLP Engineer (Product Engineering)
ASSESSMENT TYPE	Take home design and implementation exercise
DATE OF ISSUE	30 June 2026
TIME ALLOWED	72 hours from receipt

1. CONTEXT

This assessment is for an NLP Engineer role at ProITbridge AI to build system that reads, understands, and ranks candidate resumes against the requirements of a role. It is used to check how well a candidate can design, build, and explain a working solution.

2. PROBLEM STATEMENT

Design and implement a system that takes a job description and a set of candidate resumes and returns **the** resumes that best fit the role. The match shall be based on the meaning of the text, such as skills, experience, and responsibilities, and not on exact keyword matching alone.

3. OBJECTIVE

Given a job description and a collection of resumes, the system shall return the candidates whose profiles are closest to the role in meaning. The results shall be ranked, with the best-fit candidate first. The comparison shall use vector representations of the text rather than exact word overlap, and shall give a score that reflects how well each candidate matches the role.

4. FUNCTIONAL REQUIREMENTS

4.1 Resume Ingestion and Parsing

The system shall accept resumes in common formats, such as PDF, DOCX, and plain text, and extract clean, usable text from each one.

4.2 Text Preprocessing

The system shall apply the same cleaning steps, such as lowercasing, tokenising, and removing stopwords, to both the job description and all resumes.

4.3 Information Extraction

The system shall identify the key parts of a resume, such as skills, education, job titles, and years of experience, so that matching is not based on raw text alone.

4.4 Text Representation

The system shall represent the job description and resumes as dense vectors using word or sentence embeddings, such as Word2Vec, FastText, or transformer-based sentence embeddings.

4.5 Unseen and Varied Terms

The system shall handle terms that were not seen during training, as well as different ways of writing the same skill *(for example, "JS" and "JavaScript")*. FastText or subword methods may be used for this.

4.6 Matching and Ranking

The system shall measure the fit between the job description and each resume using cosine similarity, and shall return the candidates in ranked order, best fit first.

4.7 Explainability

The system shall give a short reason for each match, such as the skills or experience that caused a resume to rank highly, so that a recruiter can understand the result.

4.8 Fairness

The system shall base its ranking on skills and experience, and shall avoid using personal details such as name, gender, or age as ranking signals.

4.9 Number of Results

The system shall let the user set how many candidates are returned for each job description (the shortlist size).

5. INPUT AND OUTPUT SPECIFICATION

Input: A job description in plain text, and a set of candidate resumes to screen.

Output: A ranked shortlist of the best-fit candidates, with the closest match first, a fit score for each, and a short reason for the match.

6. SUBMISSION REQUIREMENTS

Candidates shall submit the following:

- Source code
- Report
- README
- Output screenshots
- Short explanation video

The explanation video is required. It shall cover:

(a) the overall approach and the main choices.

(b) how the system works, from resume parsing to ranking.

(c) a short run through one job description, showing how the shortlist is produced. A submission without the video will be treated as incomplete.

7. TIMELINE

The assessment shall be completed and submitted within **72 hours from the time of issue**. Late submissions will not be considered unless an extension is agreed in writing in advance.

Note to candidate: You may use any public resume or job-posting dataset and any common Python libraries. If anything is unclear, please ask before you start.